

Selenium + TestNG Interview Questions

Oct 6, 2026

These are 104 questions asked in Selenium + TestNG interviews. Every answer has the same three parts: an **explanation**, an **example** (code or a concrete case), and **In the framework**, a link to the exact file of your project, [selenium-java-framework](#), where it is done, or a note when the framework does not use it and why. The **References** section at the end lists every framework file with the questions it answers, plus the official documentation. A concrete example from your own code is the strongest answer you can give.

The repository is now an interview lab. Besides the OrangeHRM framework, it has one runnable example class per topic in [interview/](#) (locators, waits, exceptions, alerts, frames, windows, drop-downs, tables, Actions, JavaScript, cookies, uploads, TestNG and framework design); answers that have one say so under **In the lab**. The [question index](#), [common interview traps](#) and [avoid vs prefer](#) pages map short answers and common mistakes to the same code.

Level	Sections	What interviewers check
Basic	WebDriver basics, Locators, TestNG core	You know the API and the vocabulary
Intermediate	Waits, Interactions, TestNG data and execution, Maven	You can write stable tests and run them in many ways
Advanced	Framework design, Troubleshooting, Your project	You can design a framework, explain trade-offs and debug failures

Versions used in the answers: Selenium 4, TestNG 7, Java 17.

1. Selenium WebDriver basics

Q1. What is Selenium, and what are its components?

Selenium is an open source suite for automating web browsers from code. It has three parts: **WebDriver**, the API that drives a real browser; **Grid**, which spreads tests over many machines and browsers; and **IDE**, a record-and-playback browser extension. Frameworks are built on WebDriver; IDE is for quick experiments only.

Example:

```
WebDriver driver = new ChromeDriver();
driver.get("https://opensource-demo.orangehrmlive.com");
System.out.println(driver.getTitle()); // OrangeHRM
driver.quit();
```

In the framework: the whole framework uses WebDriver; [DriverManager.start\(\)](#) creates the browser. Grid and IDE are not used.

Q2. How does Selenium 4 WebDriver talk to the browser?

Your test calls the Java bindings. Each call becomes one HTTP request in the W3C WebDriver protocol, sent to the browser's driver (chromedriver, geckodriver, msedgedriver), which controls the browser and sends back the result. Selenium 4 speaks only the W3C protocol; the old JSON Wire Protocol is gone.

Example: `driver.findElement(By.name("username"))` is sent as a request to find an element with the strategy `css selector` and the value `*[name='username']`. W3C has no "name" strategy, so Selenium converts it to CSS; you can see this in error messages such as `Unable to locate element: {"method":"css selector","selector":"*[name='username']"}`.

In the framework: [DriverManager.start\(\)](#) starts `ChromeDriver`, `FirefoxDriver` or `EdgeDriver`, which launches the driver process; every page object call then goes over this protocol.

Q3. What changed in Selenium 4?

The main changes: the W3C protocol only; relative locators (`above`, `below`, `near`, `toLeftOf`); `switchTo().newWindow()` for new tabs and windows; screenshots of a single element; `getDomAttribute()` and `getDomProperty()`; waits that take a `Duration`; access to Chrome DevTools (CDP) and the BiDi protocol; and Selenium Manager, which downloads drivers.

Example:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15)); //
Duration, not seconds as long
driver.switchTo().newWindow(WindowType.TAB); // open
and switch in one step
byte[] png =
driver.findElement(By.tagName("form")).getScreenshotAs(OutputType.BYTES);
```

In the framework: the `Duration` wait in the [BasePage](#) constructor; `newWindow` in [SeleniumScenariosTest.dashboardOpensInASecondTab\(\)](#); the element screenshot in [LoginPage.formScreenshot\(\)](#); `getDomProperty` in `BasePage.valueOf()` and `getDomAttribute` in `LoginPage.placeholder()`.

Q4. What is Selenium Manager?

Selenium Manager is built into Selenium 4.6 and later. When you create a driver, it checks the installed browser version, downloads the matching driver into a cache folder if needed, and uses it. You no longer need WebDriverManager, a driver `.exe` in the project, or `System.setProperty("webdriver.chrome.driver", ...)`.

Example:

```
// Before Selenium 4.6
System.setProperty("webdriver.chrome.driver", "C:/drivers/chromedriver.exe");
WebDriver driver = new ChromeDriver();

// Now
WebDriver driver = new ChromeDriver(); // Selenium Manager finds or
downloads chromedriver
```

In the framework: `DriverManager.start()` only calls `new ChromeDriver(options)`; no driver binary is stored, and `.gitignore` even excludes `*.exe`.

Q5. What is the difference between `WebDriver` and `WebElement` ?

`WebDriver` controls the browser as a whole: open a URL, go back, switch windows, find elements, quit. `WebElement` is one element on the page: click it, type into it, read its text or properties, or search inside it.

Example:

```
WebDriver driver = DriverManager.driver();
// the browser
WebElement login =
driver.findElement(By.cssSelector("button[type='submit']")); // one element
login.click();
```

In the framework: page objects hold the `WebDriver` in a field of `BasePage`. A `WebElement` only lives inside one method (for example `BasePage.tableRows()`) and is never stored in a field, which avoids stale references.

Q6. What is the difference between `driver.close()` and `driver.quit()` ?

`close()` closes the current window or tab only; afterwards the driver points at no window until you switch to another one. `quit()` closes every window, ends the session and stops the driver process. Teardown must use `quit()`, or browser and driver processes are left running.

Example:

```

driver.close();                // close the second tab
driver.switchTo().window(firstTab); // required before the next command
// ... at the end of the test:
driver.quit();                // close everything and end the
session

```

In the framework: [DriverManager.quit\(\)](#), called by `BaseTest.stopBrowser()` after every test; `close()` plus `switchTo().window()` in [SeleniumScenariosTest.dashboardOpensInASecondTab\(\)](#).

Q7. What is the difference between `findElement` and `findElements` ?

`findElement` returns the first match and throws `NoSuchElementException` when nothing matches. `findElements` returns a list, which is empty when nothing matches, and never throws. Use `findElements` when "not there" is a valid answer.

Example:

```

// Is the error banner shown? No exception if it is not.
boolean shown = !driver.findElements(By.cssSelector(".oxd-alert-content-
text")).isEmpty();

```

In the framework: [LoginPage.loginAs\(\)](#) reads the error banner with `findElements(...).stream().findFirst().orElse("no error shown")`; [BasePage.tableRows\(\)](#) uses `findElements`, so a table without rows gives an empty list.

Q8. What is the difference between `get()` and `navigate().to()` ?

Both load a URL and wait for the page load to finish. `navigate()` also gives you the browser history: `back()`, `forward()` and `refresh()`.

Example:

```

driver.get("https://opensource-
demo.orangehrmlive.com/web/index.php/auth/login");
driver.navigate().back();
driver.navigate().refresh();

```

In the framework: [BasePage.openPath\(\)](#) uses `get()`; [LoginTest.backAfterLogoutStaysLoggedOut\(\)](#) uses `back()` and `refresh()`. Back alone showed Chrome's cached copy of the dashboard, so the test reloads to ask the server again.

Q9. How do you run tests headless, and why?

Headless means the browser runs without a window. It is faster and works on CI machines that have no display. Set the window size explicitly, because the default headless window is small and pages may switch to their mobile layout.

Example:

```
ChromeOptions options = new ChromeOptions();
options.addArguments("--headless=new", "--window-size=1920,1080");
WebDriver driver = new ChromeDriver(options);
```

In the framework: `chromeOptions()`, `firefoxOptions()` and `edgeOptions()` in [DriverManager](#) add the headless flag and 1920×1080; [Config.headless\(\)](#) reads `headless` from `config.properties`, and `mvn clean test -Dheadless=true` switches it on.

Q10. What are browser options (capabilities) used for?

Options configure the browser before it starts: headless mode, window size, language, download folder, page load strategy, and switches that turn off pop-ups or features that get in the way of tests.

Example:

```
ChromeOptions options = new ChromeOptions();
options.addArguments("--disable-features=PasswordLeakDetection"); // no
"change your password" pop-up
options.setPageLoadStrategy(PageLoadStrategy.NORMAL);
```

In the framework: [DriverManager.chromeOptions\(\)](#) disables Chrome's password-leak warning, because the warning for the well-known demo password `admin123` covered the page.

Q11. What can't Selenium test?

Selenium automates what happens inside a web page. It cannot handle CAPTCHAs (by design), desktop applications, native mobile apps (use Appium), OS dialogs such as the Windows file picker, visual layout (use a visual testing tool), load and performance (use JMeter or Gatling), or HTTP details such as status codes.

Example: to upload a file, skip the OS file picker and type the path into the `<input type='file'>` with `sendKeys` (Q38). To check a link's status code, call it with an HTTP client.

In the framework: [Links.statusCode\(\)](#) uses Java's `HttpClient` for exactly that reason: Selenium cannot read status codes (Q92).

2. Locators

Q12. Which locator strategies does Selenium support?

Eight `By` strategies: `id`, `name`, `className`, `tagName`, `linkText`, `partialLinkText`, `cssSelector` and `xpath`. Selenium 4 adds relative locators that find an element by its position next to another one (Q20).

Example:

```
By.id("login");
By.name("username");
By.className("oxd-button");
By.tagName("form");
By.linkText("Forgot your password?");
By.cssSelector("button[type='submit']");
By.xpath("//button[normalize-space()='Search']");
```

In the framework: [LoginPage](#) declares its elements with `@FindBy(name = ...)`, `@FindBy(how = How.CSS, using = ...)` and `@FindBy(tagName = "form")`; [BasePage](#) uses CSS for OrangeHRM widgets and builds `By.xpath` locators where text or parent elements are needed.

Q13. Which locator do you prefer, and why?

Prefer what is least likely to change: a stable `id`, `name` or test attribute such as `data-testid` first, then a short CSS selector, then XPath when you need text or parents. Avoid what DevTools' "Copy XPath" gives you: it is positional and breaks easily.

Example: for the user name field, `By.name("username")` beats

`By.xpath("//div[2]/div/form/div[1]/div/div[2]/input")`; both find the same field today, but only the first survives a layout change.

In the framework: `@FindBy(name = "username")` and `@FindBy(name = "password")` in [LoginPage](#), `@FindBy(name = "firstName")` in [AddEmployeePage](#); CSS on OrangeHRM's stable `oxd-*` classes; XPath only for text and labels.

Q14. CSS selector or XPath: what is the difference?

CSS is shorter, often faster and easy to read, but it can only move down the DOM and cannot match visible text. XPath can do both: move up to parents (`ancestor::`, `..`), across to siblings, and match text (`normalize-space()='Save'`).

Example: the same Search button both ways, and one only XPath can express:

```
By.cssSelector("form button[type='submit']"); // by attributes only
By.xpath("//button[normalize-space()='Search']"); // by its
visible text
By.xpath("//label[normalize-space()='Username']/ancestor::div[1]"); // up to
a parent: XPath only
```

In the framework: CSS for widgets in [BasePage](#) (the `@FindBy(css = ...)` fields `toastMessage`, `tableRowElements`, `dropdownOptions`); XPath in `BasePage.button(text)` (visible text) and `BasePage.inField(label, inner)` (up to the field's container).

Q15. What is the difference between absolute and relative XPath?

Absolute XPath starts at the root (`/html/body/div[2]/...`) and lists every step, so any change to the page layout breaks it. Relative XPath starts with `//` and targets attributes or text, so it keeps working when the layout changes. A framework should contain no absolute XPath.

Example:

```
By.xpath("/html/body/div/div[1]/div/div[1]/div/div[2]/div[2]/form/div[3]/butt
on"); // absolute: brittle
By.xpath("//button[@type='submit']");
// relative: stable
```

In the framework: every XPath is relative; see `button()`, `inField()` and `choose()` in [BasePage](#).

Q16. Name some useful XPath functions and axes.

Functions: `contains()`, `starts-with()`, `normalize-space()` (trims and collapses spaces) and `text()`. Axes: `ancestor`, `parent`, `following-sibling`, `preceding-sibling` and `descendant` (`./`).

Example:

```
By.xpath("//div[contains(@class,'oxd-table-card')]"); //
contains()
By.xpath("//a[starts-with(@href,'/web/index.php/pim')]"); //
starts-with()
By.xpath("//label[normalize-space()='Username']/ancestor::div[1]"); //
normalize-space() + ancestor
By.xpath("//label[text()='Status']/../following-sibling::div"); //
parent + following-sibling
```

In the framework: `normalize-space()` in `BasePage.button()`; `ancestor::` in `BasePage.inField()`; `contains(@class, ...)` throughout `BasePage`; `./` (descendant) in `EmployeeListPage.delete()`.

In the lab: `LocatorStrategiesTest.xpathAxesNavigateFromALabel()` goes from a label to its input with `following-sibling::` and up to the form with `ancestor::`.

Q17. How do you handle elements whose id changes on every load?

Locate by something that does not change: a fixed part of the attribute (`contains`, `starts-with`), the visible text, a nearby label, or a `data-*` attribute. Where nothing is stable, ask the developers to add test ids.

Example: an input with `id="user_8f3a2"` that changes every load:

```
By.xpath("//input[starts-with(@id,'user_')]);
By.cssSelector("input[id^='user_']"); // the same in CSS
```

In the framework: OrangeHRM inputs have no stable ids, so fields are found by their visible label with `BasePage.inField()`, and table rows by the text they contain (Q19).

In the lab: `LocatorStrategiesTest.dynamicLocatorIsBuiltFromAValue()` fills one XPath template with `String.format` for any e-mail address in a table.

Q18. How do you find an input when the label is not linked to it?

Normally a `<label for="...">` points at its input. When it does not, start at the label text, go up to the container that holds both label and input, then down to the input.

Example:

```
By.xpath("//label[normalize-space()='Employee Id']"
+ "/ancestor::div[contains(@class,'oxd-input-group')]/input");
```

In the framework: `BasePage.inField(label, inner)` builds this XPath, and `input(label)`, `selectField(label)` and `fieldError(label)` reuse it, for example `input("Username")` in `SystemUsersPage.filterByUsername()`.

Q19. How do you find a row in a table by its content, and click something in it?

Find the row that contains a cell with the text you want, then go down from that row to the button. To read a table, map each cell to its column title, so the test never depends on column positions.

Example: click the delete button in the row of employee `QA12345678`:

```
By.xpath("//div[contains(@class,'oxd-table-card')]"+
    "[./div[contains(@class,'oxd-table-cell')][normalize-
space()='QA12345678']]"+
    "//button[./i[contains(@class,'bi-trash')]]");
```

In the framework: `EmployeeListPage.delete(employeeId)` uses this XPath; `BasePage.tableRows()` reads every row as a map of column title → text, and `column("Username")` returns one column.

Q20. What are relative locators?

Selenium 4 locators that find an element by where it is on screen compared with another element: `above`, `below`, `toLeftOf`, `toRightOf` and `near`. They help when the DOM gives no good hook, but they depend on the layout, so use them sparingly.

Example:

```
import static org.openqa.selenium.support.locators.RelativeLocator.with;

WebElement password = driver.findElement(
    with(By.tagName("input")).below(By.name("username")));
```

In the framework: the page objects don't need them: finding fields by their label (`BasePage.inField()`) does not depend on screen position, so it also works in headless mode and at any window size. **In the lab:** `LocatorStrategiesTest.relativeLocatorsFindFieldsByPosition()` finds the password field `below` the user name field and the "Password" label `above` the password field.

3. Waits and synchronization

Wait	Set on	Waits for	Use it when
Implicit	The driver, once	Any element to be present, on every <code>findElement</code>	Rarely; it hides timing and mixes badly with explicit waits
Explicit (<code>WebDriverWait</code>)	One action	A condition: visible, clickable, URL, text	Default choice for every action
Fluent (<code>FluentWait</code>)	One action	A condition, with your own polling interval and ignored exceptions	Custom polling or exceptions to ignore

Q21. Why should you avoid `Thread.sleep()` ?

A sleep always waits the full time. When the page is fast the test wastes time; when the page is slow the sleep is too short and the test fails. Tests end up both slow and flaky. An explicit wait returns as soon as the condition is true and only waits longer when it has to.

Example:

```
Thread.sleep(5000); // always 5 s, and still fails if
the page needs 6 s
driver.findElement(SEARCH).click();

new WebDriverWait(driver, Duration.ofSeconds(15))
    .until(ExpectedConditions.elementToBeClickable(SEARCH)).click(); //
as fast as the page allows
```

In the framework: there is no `Thread.sleep` anywhere. Every action in [BasePage](#) (`click`, `type`, `waitVisible`, `textOf`) goes through an explicit wait with the timeout from `config.properties` (`timeoutSeconds=15`).

Q22. Why not mix implicit and explicit waits?

The implicit wait applies inside every `findElement` call, including the ones the explicit wait makes on each poll. The two timeouts then add up in ways that are hard to predict, and a check for "element is gone" can take the full implicit time on every poll. Selenium's own documentation warns against mixing them.

Example: with `implicitlyWait(10 s)` and an explicit wait of 5 s for a spinner to disappear, a single poll can already block for up to 10 s, so the 5 s wait may take 10 s or more.

In the framework: no implicit wait is ever set in [DriverManager](#); only the explicit waits of [BasePage](#) are used.

In the lab: [ImplicitWaitTest](#) shows a lookup failing without any wait, passing with an implicit wait, and the wait reset to zero afterwards.

Q23. Name some common `ExpectedConditions` .

The most used ones: `visibilityOfElementLocated`, `elementToBeClickable`, `presenceOfElementLocated`, `invisibilityOfElementLocated`, `visibilityOfAllElementsLocatedBy`, `urlContains`, `textToBePresentInElement`, `alertIsPresent` and `frameToBeAvailableAndSwitchToIt`.

Example:

```
wait.until(ExpectedConditions.urlContains("/dashboard"));
wait.until(ExpectedConditions.invisibilityOfElementLocated(By.cssSelector(".o
xd-loading-spinner")));
```

In the framework: [BasePage](#) uses `elementToBeClickable (click, type)`, `visibilityOf` and `visibilityOfElementLocated (waitVisible)`, `urlContains (waitForUrl)`, `invisibilityOfAllElements (waitForLoader)`, and `visibilityOfAllElements (textsOf)`, used by `options`, `validationErrors` and `AppPage.menuItems`. The conditions that take a `WebElement` work with the page's `@FindBy` fields, because each call looks the element up again.

Q24. What is the difference between presence and visibility?

Presence means the element is in the DOM. Visibility means it is in the DOM **and** displayed with a size greater than zero. You can only click, type into or read the text of a visible element; presence is enough to know that something has started, such as a spinner.

Example: a hidden error message is present but not visible, so

`presenceOfElementLocated` passes while `visibilityOfElementLocated` keeps waiting.

In the framework: [BasePage.waitForLoader\(\)](#) uses presence to notice that the spinner has appeared (the `@FindBy` list `loaders` is no longer empty); `waitVisible()` uses visibility before reading text.

In the lab: [ExplicitWaitTest.presenceIsNotVisibility\(\)](#): the hidden result is present at once, but only the visibility wait waits until it is shown.

Q25. What is a `StaleElementReferenceException`, and how do you fix it?

A `WebElement` points at one node in the DOM. If the page removes or re-renders that node, the reference goes stale, and using it throws this exception. Fix it by finding the element again just before you use it, by not storing elements in fields, and by letting waits retry.

Example:

```
WebElement firstRow = driver.findElement(TABLE_ROWS);
search();                // the table re-renders
firstRow.getText();      // StaleElementReferenceException

wait.ignoring(StaleElementReferenceException.class); // a wait then simply
polls again
```

In the framework: the `BasePage` constructor calls

`wait.ignoring(StaleElementReferenceException.class)`, and `tableRows()` reads the whole table inside one `wait.until(...)`, so a re-render in the middle of reading just retries. This fixed a real flaky search test. The page's `@FindBy` fields do not go stale either: each is a proxy that finds the element again on every call (unless `@CacheLookup` is added, see Q42).

In the lab: `SeleniumExceptionsTest.staleElementReferenceException()` removes a checkbox, waits with `stalenessOf`, shows the old reference throwing, then finds the new checkbox again.

Q26. How do you wait for a loading spinner?

Wait for it to disappear. The catch: if you check before the spinner has appeared, the "invisible" check passes at once and the test reads the old page. So first give the spinner a short moment to appear, then wait for it to go.

Example:

```
@FindBy(css = ".oxd-loading-spinner, .oxd-form-loader")
private List<WebElement> loaders;           // empty while no spinner is
shown

try {
    new WebDriverWait(driver, Duration.ofSeconds(2)).until(d ->
!loaders.isEmpty()); // started?
} catch (TimeoutException e) {
    // the response was faster than the spinner
}
wait.until(ExpectedConditions.invisibilityOfAllElements(loaders));
// finished
```

In the framework: `BasePage.waitForLoader()`, called after every search, reset, delete and page load in `SystemUsersPage`, `EmployeeListPage` and `EmployeeProfilePage`.

Q27. How do you write a custom wait condition?

Pass a lambda to `wait.until(...)`. It receives the driver; return `null` or `false` to keep polling, or a value to finish, which `until` then returns.

Example: wait until a header has text, and return that text:

```
String name = wait.until(d -> {
    String text = fullNameHeader.getText().trim();    // @FindBy field, looked
    up on each poll
    return text.isEmpty() ? null : text;                // null = not yet, keep
    polling
});
```

In the framework: `EmployeeProfilePage.fullName()` is exactly this, because the header is drawn before the name arrives; `BasePage.tableRows()` is a larger custom wait that returns the whole table.

In the lab: `FluentWaitTest` builds a `FluentWait` by hand (timeout, polling, ignored exception, message) and a custom condition that returns the value it waited for.

Q28. What happens when an explicit wait times out?

It throws `TimeoutException`. The message names the condition and the locator, which is the first clue when you debug. Where "not shown" is a valid answer, catch it and return `false`; where it hides the real reason, replace it with a clearer error.

Example: Expected condition failed: waiting for visibility of element found by By.name: username (tried for 15 seconds with 500 milliseconds interval).

In the framework: `isVisible()` and `waitForUrl()` in `BasePage` catch it and return `false`; `LoginPage.loginAs()` throws `Login as "Admin" failed: Invalid credentials` instead of a 15-second timeout on the dashboard.

In the lab: `SeleniumExceptionsTest.timeoutException()` lets a 2-second wait run out and checks that the message names the condition.

4. WebDriver interactions

Q29. How do you handle a `<select>` dropdown?

Wrap the element in Selenium's `Select` class. It can pick an option by its visible text, its `value` attribute or its position, and read the current choice. It only works on real `<select>` elements.

Example:

```
Select country = new Select(driver.findElement(By.id("country")));
country.selectByVisibleText("India");
country.selectByValue("IN");
country.selectByIndex(2);
String chosen = country.getFirstSelectedOption().getText();
```

In the framework: OrangeHRM has no `<select>` elements; its drop-downs are custom div s, handled as in Q30. **In the lab:** `SelectDropDownTest` uses `Select` on the practice site: by visible text, value and index, reading every option, and a disabled option being refused with `UnsupportedOperationException`.

Q30. How do you handle a custom dropdown that is not a `<select>` ?

`Select` throws `UnexpectedTagNameException` on anything that is not a `<select>`. Treat a custom drop-down like a user would: click the box to open the list, then click the option by its text. To read all options, open the list, read the option elements and close it again.

Example:

```
click(inField("User Role", "//div[contains(@class,'oxd-select-text')]"));
// open
click(By.xpath("//div[@role='listbox']//div[@role='option'][normalize-
space()='Admin']")); // pick
```

In the framework: `BasePage.choose(label, option)` picks an option, `BasePage.options(label)` reads them all and `BasePage.chosen(label)` reads the current one; used by `SystemUsersPage.filterByRole()` and checked in `SeleniumScenariosTest.userRoleDropdownOffersAdminAndEss()`.

Q31. How do you handle JavaScript alerts?

Browser alerts (`alert`, `confirm`, `prompt`) are not part of the page, so `findElement` cannot see them. Switch to the alert, then accept, dismiss, read its text or type into a prompt. Wait for it first, because it may appear a moment after the click.

Example:

```
Alert alert = wait.until(ExpectedConditions.alertIsPresent());
String text = alert.getText();
alert.accept();           // OK;  alert.dismiss() = Cancel;
alert.sendKeys("x") for a prompt
```

In the framework: OrangeHRM shows an in-page dialog, not a browser alert, when you delete an employee. Because it is normal HTML, `EmployeeListPage.delete()` simply clicks its "Yes, Delete" button (the `confirmDeleteButton` field). Knowing which kind of dialog you face is the real interview point. **In the lab:** `AlertHandlingTest` handles a real alert, confirm and prompt; `SeleniumExceptionsTest.alertExceptions()` shows `NoAlertPresentException` and `UnhandledAlertException`.

Q32. How do you work with iframes?

An iframe is a page inside the page; elements inside it cannot be found until you switch into it. Switch by index, by name or id, or by the frame element; work inside; then switch back with `defaultContent()` (the top page) or `parentFrame()` (one level up).

Example:

```
wait.until(ExpectedConditions.frameToBeAvailableAndSwitchToIt(By.id("editor")
));
driver.findElement(By.tagName("body")).sendKeys("Hello");
driver.switchTo().defaultContent();
```

In the framework: the OrangeHRM pages under test have no iframes. **In the lab:**

[IframeHandlingTest](#) switches into nested frames by name, goes up with `parentFrame()`, back with `defaultContent()`, by index, and into an editor iframe by its `WebElement`.

Q33. How do you switch between windows or tabs?

Every window or tab has a handle (an id string). Store the current handle before you open anything, find the new handle among `getWindowHandles()`, switch to it, and switch back at the end. Selenium 4's `newWindow()` opens a new tab or window and switches to it in one step.

Example:

```
String first = driver.getWindowHandle();
link.click();
for (String handle : driver.getWindowHandles()) {
    if (!handle.equals(first)) {
        driver.switchTo().window(handle);
    }
}
// ... work in the new tab ...
driver.close();
driver.switchTo().window(first);
```

In the framework: [SeleniumScenariosTest.dashboardOpensInASecondTab\(\)](#) opens a tab with `newWindow(WindowType.TAB)`, checks it shares the login session, closes it and switches back (Q95).

In the lab: [MultipleWindowTest](#) finds a window the page opened by comparing the handle sets before and after the click, and by its title.

Q34. When do you use the `Actions` class?

For input that a single `click()` or `sendKeys()` cannot express: `hover (moveToElement)`, `right-click (contextClick)`, `double-click`, `drag and drop`, `click-and-hold`, and `key sequences or chords`. Build a chain, then call `perform()`; nothing happens before that.

Example:

```
new Actions(driver).moveToElement(menu).perform();           // hover to
open a menu
new Actions(driver).dragAndDrop(card, column).perform();     // drag and
drop
new
Actions(driver).keyDown(Keys.CONTROL).click(row).keyUp(Keys.CONTROL).perform(
); // Ctrl+click
```

In the framework: `LoginPage.loginWithKeyboard()` logs in with one `Actions` chain (type, Tab, type, Enter); tested by `SeleniumScenariosTest.adminCanLogInWithTheKeyboard()` (Q94).

In the lab: `ActionsClassTest` hovers to reveal a caption, right-clicks, holds Shift while typing, and moves a slider with the arrow keys.

Q35. When do you use `JavascriptExecutor`, and what is the risk?

Use it for what the WebDriver API cannot do: read page state (`document.readyState`), scroll an element into view, read values the API does not expose, or as a last resort click an element that a normal click cannot reach. The risk: a JavaScript click skips the checks a real user's click goes through (visible, enabled, not covered), so it can hide real bugs.

Example:

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("arguments[0].scrollIntoView(true);", element);
String state = (String) js.executeScript("return document.readyState;");
```

In the framework: `BasePage.runScript(script, args...)`, with a Javadoc warning about JavaScript clicks; used by `AppPage.readyState()` and `AppPage.titleFromScript()` (Q96). No test clicks through JavaScript.

In the lab: `JavaScriptExecutorTest` shows arguments and the Java types of returned values, scrolling to load more content, and a JavaScript click going through an overlay that a real click is refused by.

Q36. `clear()` does not empty a field. Why, and what do you do?

In React and Vue apps the page keeps the field's value in its own form model, which listens for keyboard events. `clear()` changes the DOM value without those events, so the model keeps the old value and puts it back, or submits it. Clear the field the way a

user would: select all and delete.

Example:

```
field.sendKeys(Keys.chord(Keys.CONTROL, "a"), Keys.DELETE); // Keys.COMMAND
on macOS
field.sendKeys("new value");
```

In the framework: [BasePage.type\(locator, text\)](#) does exactly this, because OrangeHRM's Vue forms kept the old value after `clear()`.

Q37. How do you take a screenshot?

Cast the driver to `TakesScreenshot` and choose the output: `FILE` (a temporary file), `BYTES` or `BASE64`. Selenium 4 can also capture a single `WebElement` (Q97). Take it on failure, before the browser closes, so it shows the moment of failure.

Example:

```
byte[] png = ((TakesScreenshot) driver).getScreenshotAs(OutputType.BYTES);
Files.write(Path.of("target", "screenshots", "failure.png"), png);
```

In the framework: [BaseTest.attachScreenshot\(\)](#) captures Base64 when a test fails, saves a PNG to `target/screenshots`, and stores it on the TestNG result; [HtmlReportListener](#) embeds it in the report.

Q38. How do you upload a file?

Do not click the upload button: that opens the operating system's file picker, which Selenium cannot control. Instead call `sendKeys` with the file's full path on the `<input type='file'>` element, even if it is hidden behind a styled button. On Selenium Grid, set a `LocalFileDetector` so the file is copied to the remote machine.

Example:

```
WebElement upload = driver.findElement(By.cssSelector("input[type='file']"));
upload.sendKeys(Path.of("src/test/resources/photo.png").toAbsolutePath().toString());
// On Grid: ((RemoteWebDriver) driver).setFileDetector(new
LocalFileDetector());
```

In the framework: the OrangeHRM tests upload nothing; the Add Employee photo would be the place for it. **In the lab:** [FileUploadTest](#) creates a file, calls `sendKeys` with its absolute path on the file input, submits, and checks the name the site reports.

Q39. How do you read text, attributes and properties?

`getText()` returns the visible text. `getDomAttribute("x")` returns the attribute as written in the HTML; `getDomProperty("x")` returns the live value from the DOM, such as what the user typed into an input. The older `getAttribute()` mixes both and is best avoided in new code.

Example:

```
label.getText(); // "Username"
input.getDomAttribute("placeholder"); // "Username" (from the HTML)
input.getDomProperty("value"); // "Admin" (what was typed)
link.getDomProperty("href"); // absolute URL, even if the HTML has
a relative one
```

In the framework: `BasePage` `textOf()` (`getText`) and `valueOf()` (`getDomProperty("value")`); `LoginPage.placeholder()` (`getDomAttribute`); `AppPage.menuLinks()` (`getDomProperty("href")`).

5. Page Object Model and framework design

Q40. What is the Page Object Model, and why use it?

Each page (or part of a page) gets a class that holds its locators and offers its actions as methods. Tests call those methods and only assert. When the UI changes, you fix one page class instead of every test, and tests read like the scenario rather than like Selenium code.

Example:

```
// Without page objects: locators and waits inside the test
driver.findElement(By.name("username")).sendKeys("Admin");
driver.findElement(By.name("password")).sendKeys("admin123");
driver.findElement(By.cssSelector("button[type='submit']")).click();

// With page objects: the test says what, the page knows how
DashboardPage dashboard = new LoginPage(driver).open().loginAs("Admin",
"admin123");
```

In the framework: eight page classes in `pages/`, such as `LoginPage` and `SystemUsersPage`. No test class contains a locator.

Q41. Should page objects contain assertions?

Generally no. A page returns data or the next page; the test decides what is right. That keeps pages reusable: the same `errorMessage()` serves a test that expects "Invalid credentials" and one that expects no error. The exception is a check that the page itself

loaded, which belongs in the page.

Example:

```
// Page: returns what it sees
public String errorMessage() { return textsOf(errorAlerts).get(0); }

// Test: decides what is right
assertEquals(login.errorMessage(), "Invalid credentials");
```

In the framework: pages return values such as `LoginPage.errorMessage()` or `SystemUsersPage.usernames()`; only the test classes call `assertEquals`. Load checks such as `waitUntilLoaded()` stay in the pages.

Q42. What is Page Factory, and do you use it?

Page Factory is Selenium's way to declare a page's elements as fields with `@FindBy` annotations. `PageFactory.initElements(driver, this)` fills each field with a lazy proxy that finds the element again on every call, so a page that re-renders does not leave it stale. `@CacheLookup` makes the proxy keep the first element it found: fewer lookups, but stale after a reload or re-render. Locators built from a value (a field label, a row ID) cannot be annotations and stay methods that return a `By`.

Example:

```
@FindBy(name = "username")
private WebElement usernameField;           // found again on every call

@FindBy(name = "username")
@CacheLookup
private WebElement cachedUsernameField;      // keeps the first element:
stale after a reload

@FindBy(css = ".oxd-alert-content-text")
private List<WebElement> errorAlerts;        // empty list when there is no
banner

PageFactory.initElements(driver, this);      // in the BasePage constructor
```

In the framework: yes. The `BasePage` constructor calls `PageFactory.initElements(driver, this)`, and every page declares its fixed elements as `@FindBy` fields, for example `LoginPage`. The `BasePage` helpers (`click`, `type`, `textOf`, `textsOf`, `waitVisible`) take those fields, so every use still runs inside an explicit wait.

[SeleniumScenariosTest.findByFieldSurvivesAReloadButCacheLookupGoesStale\(\)](#)

shows the difference: after a reload the plain field still works, and the `@CacheLookup` field throws `StaleElementReferenceException`. `@CacheLookup` is used nowhere else.

Q43. What is method chaining (a fluent page API)?

Each method returns the page you are on after the action, so steps chain into one readable line. Navigation becomes explicit, and the compiler stops you from calling a method on the wrong page: after `loginAsAdmin()` you have a `DashboardPage`, which has no `submit()`.

Example:

```
SystemUsersPage users = loginAsAdmin() // LoginPage → DashboardPage
    .openAdmin()                       // DashboardPage →
SystemUsersPage
    .filterByUsername("Admin")         // returns this
    .search();                         // returns this
```

In the framework: [SystemUsersPage](#) filter methods return `this`; [AppPage.openAdmin\(\)](#) returns the next page; see [SystemUsersTest](#).

Q44. What goes into a `BasePage` ?

What every page needs: the driver, the wait, and safe versions of the basic actions (click, type, read text, wait for a URL). Widgets that the application uses everywhere also belong there, so each page is only a few lines. Pages after login share a second parent for the menus.

Example:

```
public abstract class BasePage {
    protected final WebDriver driver;
    protected final WebDriverWait wait;

    protected void click(By locator) {
        wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
    }
}
```

In the framework: [BasePage](#) holds the waits plus OrangeHRM's widgets: `openPath`, `inField`, `button`, `choose`, `options`, `tableRows`, `toast`, `validationErrors` and `waitForLoader`. [AppPage](#) adds the side menu and user menu for every page after login.

Q45. How do you make the driver safe for parallel runs?

Keep one driver per thread in a `ThreadLocal<WebDriver>`, never in a static field. Each thread then only ever sees its own browser. Call `remove()` after `quit()`, because test runners reuse threads.

Example:

```
private static final ThreadLocal<WebDriver> DRIVER = new ThreadLocal<>();

public static WebDriver driver() { return DRIVER.get(); }

public static void quit() {
    try { DRIVER.get().quit(); } finally { DRIVER.remove(); }
}
```

In the framework: [DriverManager](#) does this; `start()` also quits any browser the thread still had open, so none are leaked.

Q46. How do you manage configuration such as URL, browser and timeouts?

Keep defaults in a properties file on the classpath, and let a system property with the same key win. CI then changes values with `-Dkey=value`, without code changes or a second file.

Example:

```
String value = System.getProperty(key, properties.getProperty(key)); // -D
wins, else the file
```

```
mvn clean test -Dbrowser=edge -Dheadless=true -DtimeoutSeconds=20
```

In the framework: [Config](#) reads [config.properties](#) once and fails with a clear message when a key is missing.

Q47. Where do you keep test data?

Outside the test logic: data providers for cases, properties or JSON for fixed values, and generated values for anything a test creates. Generated data must be unique, so parallel runs and reruns never collide on a shared environment.

Example:

```
String number = String.format("%08d",
    ThreadLocalRandom.current().nextInt(100_000_000));
Employee employee = new Employee("Selenium", "Test" + number, "QA" + number);
// unique every run
```

In the framework: `TestData` holds the `invalidCredentials` data provider and `newEmployee()`; the ID is `QA` + 8 digits, exactly the 10 characters the Employee Id field allows.

Q48. How do you keep tests independent?

Each test gets its own browser and data, logs in itself, cleans up what it created, and never relies on another test's result or order. Then any test can run alone, in any order, or in parallel.

Example: instead of test 1 creating an employee and test 2 deleting it, one test creates, checks and deletes its own employee.

In the framework: `BaseTest` starts a new browser in `@BeforeMethod`; every test logs in itself; `EmployeeTest` deletes its own employee and has a `cleanUp()` for failed runs. Checks never count records on the shared demo site.

Q49. What design patterns are used in Selenium frameworks?

Page Object (one class per page), **Factory** (create a driver from a browser name), **Singleton** (one shared configuration), **Fluent interface** (chained page methods), **Template method** (a base class defines the steps, subclasses fill one in) and **ThreadLocal** per-thread state.

Example: a factory method: `switch (browser) { case "chrome" -> new ChromeDriver(...); case "edge" -> new EdgeDriver(...); }.`

In the framework: Factory in `DriverManager.start()`; Singleton in `Config.get()`; Fluent interface in the pages; Template method in `BaseTest.stopBrowser()`, which calls the `cleanUp()` hook that `EmployeeTest` overrides.

Q50. How do you log test steps?

Record each user action as a readable step in the report, not only the failure. When a test fails, the steps show how far it got without rerunning it. Mask secrets such as passwords.

Example:

```
Steps.log("Log in with username \"Admin\" and password \"*****\""); //
goes to Reporter.log
```

In the framework: `Steps.log()` wraps TestNG's `Reporter.log`; page methods call it (passwords masked in `LoginPage.mask()`), and `HtmlReportListener` lists them per test.

6. TestNG core

Q51. What is TestNG, and why use it for UI tests?

TestNG is a Java test framework. Out of the box it offers groups, dependencies between tests, data providers, parallel runs, listeners, retries and XML suite files. JUnit 5 has caught up on many of these, but TestNG's groups, `dependsOnMethods` and suite files are still common in Selenium projects.

Example:

```
@Test(groups = "smoke", description = "Admin can log in and sees the
dashboard")
public void adminCanLogIn() {
    assertTrue(loginAsAdmin().isDisplayed());
}
```

In the framework: TestNG 7.12 runs every test (see [pom.xml](#)); groups, data providers, parallel classes, listeners and a retry analyzer are all used.

Q52. In what order do TestNG annotations run?

Configuration methods wrap each level of the suite, from the outside in, and close in reverse order:

1. `@BeforeSuite`
2. `@BeforeTest` (once per `<test>` tag in testng.xml)
3. `@BeforeClass`
4. `@BeforeMethod`
5. `@Test`
6. `@AfterMethod`
7. `@AfterClass`
8. `@AfterTest`
9. `@AfterSuite`

`@BeforeGroups` and `@AfterGroups` run before the first and after the last test of a group. With inheritance, a parent's `@Before` method runs before the child's, and a child's `@After` method before the parent's.

Example: with 2 test methods in a class: BeforeClass, BeforeMethod, test 1, AfterMethod, BeforeMethod, test 2, AfterMethod, AfterClass.

In the framework: [BaseTest](#) uses `@BeforeMethod` to start a browser and `@AfterMethod` to take a screenshot, clean up and quit.

In the lab: [AnnotationOrderTest](#) records every configuration annotation as it runs and checks the order; the full order is printed at the end of the suite.

Q53. What is the difference between `@BeforeTest` and `@BeforeMethod` ?

`@BeforeTest` runs once per `<test>` block in `testng.xml`, before all classes in it, despite its name. `@BeforeMethod` runs before every single `@Test` method, and before every row of a data provider.

Example: a class with 3 tests and one 3-row data provider test gets 6 `@BeforeMethod` calls but only 1 `@BeforeTest` call.

In the framework: `BaseTest.startBrowser()` is a `@BeforeMethod`, so every test, and every data provider row, gets a fresh browser.

Q54. What does `alwaysRun = true` do on a configuration method?

Two things: the method still runs when you filter by groups (a `@BeforeMethod` without a group would otherwise be skipped by `-Dgroups=smoke`), and an `@After` method still runs after a failure or skip. Without it, a group run can start no browser, or a failed test can leave one open.

Example:

```
@AfterMethod(alwaysRun = true)
public void stopBrowser(ITestResult result) { ... } // runs after every
test, whatever happened
```

In the framework: `startBrowser()` and `stopBrowser()` in `BaseTest` both use it, so `mvn test -Dgroups=smoke` still starts and closes browsers.

Q55. What are groups, and how do you run them?

Groups are labels on tests, such as `smoke` or `regression`. A test can have several. You choose groups to run or exclude in `testng.xml` or on the command line; a group set on the class applies to all its methods.

Example:

```
@Test(groups = "login") // class level: every test is
in "login"
public class LoginTest extends BaseTest {
    @Test(groups = "smoke") // this one is in "login" and
    "smoke"
    public void adminCanLogIn() { ... }
}
```

```
mvn clean test -Dgroups=smoke
```

In the framework: the groups are `smoke`, `login`, `admin`, `pim`, `interview` and `demo` (see `LoginTest`); `demo` is excluded by default in `pom.xml` and `testng.xml`.

Q56. What do `priority`, `enabled`, `timeOut` and `invocationCount` do?

`priority` orders tests within a class (lower runs first; the default is 0). `enabled = false` turns a test off without deleting it. `timeOut` fails a test that runs longer than the given milliseconds. `invocationCount` runs the same test several times.

Example:

```
@Test(priority = 1)                public void loginPageIsShown() { ... }
@Test(enabled = false)             public void notReadyYet() { ... }
@Test(timeOut = 30_000)           public void finishesInTime() { ... }
@Test(invocationCount = 2)        public void
wrongPasswordIsRejectedEveryTime() { ... }
```

In the framework: `priority` and `invocationCount` in [TestNgFeaturesTest](#). `timeOut` in `TestNgFeaturesTest.loginFinishesWithinOneMinute()` (Q104).

Q57. Should tests rely on `priority` to run in order?

No. Ordered tests depend on each other: one failure breaks the rest, and you cannot run one test alone. Make each test independent. Use `priority` only for a sensible run order, and `dependsOnMethods` only when one test really needs another to pass first.

Example: "test 1 creates a user, test 2 edits it, test 3 deletes it" fails all three when test 1 fails; one test that creates, edits and deletes its own user does not.

In the framework: every test logs in and creates its own data; `priority` appears only in the demonstration in [TestNgFeaturesTest](#) (Q100).

Q58. What does `dependsOnMethods` do when the first test fails?

The dependent test is **skipped**, not failed, and the report says which method it depended on. That keeps one root failure from showing up as many failures. `alwaysRun = true` on the dependent test would run it anyway.

Example:

```
@Test public void dashboardTitleCheckFails() { ... }           // fails

@Test(dependsOnMethods = "dashboardTitleCheckFails")
public void skippedBecauseDependencyFailed() { ... }           // skipped, never
starts
```

In the framework: [ReportDemoTest.skippedBecauseDependencyFailed\(\)](#) shows this on purpose; the report names the failed method.

In the lab: [DependencyTest](#) shows `dependsOnGroups` and `enabled = false`.

Q59. What is the difference between hard and soft assertions?

A hard assertion (`Assert`) stops the test at the first failure. A soft assertion (`SoftAssert`) records each failure and carries on; `assertAll()` at the end fails the test with every mismatch listed. Forget `assertAll()` and the test passes even though checks failed.

Example:

```
SoftAssert soft = new SoftAssert();
soft.assertEquals(title, "OrangeHRM", "Title");
soft.assertEquals(buttonText, "Login", "Button");
soft.assertAll();           // fails here, listing both if both are wrong
```

In the framework: hard asserts in every functional test; `SoftAssert` in [TestNgFeaturesTest.softAssertChecksTheLoginPage\(\)](#) (Q98).

Q60. How do you skip a test at runtime?

Throw `SkipException("reason")` when a precondition is missing, for example a feature that is off in this environment or a browser that is not supported. The test is reported as skipped with your reason, not as failed.

Example:

```
if (!Config.get().browser().equals("chrome")) {
    throw new SkipException("Only runs on Chrome");
}
```

In the framework: [ReportDemoTest.skippedWhenPreconditionIsMissing\(\)](#) skips because the demo site sends no e-mails; the report shows the steps before the skip and the reason.

Q61. How do you test that an exception is thrown?

Use `expectedExceptions` on the `@Test`, optionally with a regular expression for the message, or `Assert.assertThrows(...)` inside the test when only one line should throw.

Example:

```
@Test(expectedExceptions = IllegalStateException.class,
      expectedExceptionsMessageRegExp = ".*Invalid credentials.*")
public void wrongPasswordThrowsTheLoginError() { ... }

assertThrows(IllegalArgumentException.class, () ->
    DriverManager.start("safari", true));
```

In the framework: [TestNgFeaturesTest.wrongPasswordThrowsTheLoginError\(\)](#) (Q99). `DriverManager.start()` really throws `IllegalArgumentException` for an unknown browser.

7. TestNG data, suites, parallel runs and listeners

Q62. What is a `@DataProvider` ?

A method that returns `Object[][]` (or an `Iterator<Object[]>`). TestNG runs the test once per row and passes the row's values as the test method's parameters. Each row is reported as its own result, so one bad row does not hide the others.

Example:

```
@DataProvider(name = "invalidCredentials")
public static Object[][] invalidCredentials() {
    return new Object[][] {
        { "wrong password", "Admin", "wrong-password" },
        { "unknown user", "no.such.user", "admin123" },
    };
}

@Test(dataProvider = "invalidCredentials")
public void loginIsRejected(String caseName, String username, String
password) { ... }
```

In the framework: [TestData.invalidCredentials\(\)](#) has 3 rows, so [LoginTest.loginIsRejectedForInvalidCredentials\(\)](#) runs 3 times; the report shows each case by name (wrong password, unknown user, wrong case).

In the lab: [DataProviderVariantsTest](#) shows the other forms: a lazy `Iterator<Object[]>`, a provider that takes the calling `Method` and serves two tests, and `parallel = true`.

Q63. How do you share a data provider between test classes?

Put it in its own class as a `static` method, and point to it with `dataProviderClass`. Several test classes can then use the same data without copying it.

Example:

```
@Test(dataProvider = "invalidCredentials", dataProviderClass =
TestData.class)
```

In the framework: exactly this line in [LoginTest](#); the provider lives in [TestData](#).

Q64. What is the difference between `@DataProvider` and `@Parameters` ?

`@Parameters` reads single values from `testng.xml`, such as a browser or URL for the whole suite: one value, one run. `@DataProvider` feeds many rows to one test: one run per row. `@DataProvider(parallel = true)` also runs the rows in parallel.

Example: `@Parameters("browser")` with `<parameter name="browser" value="chrome"/>` runs once on Chrome; a data provider with 3 browsers runs the test 3 times.

In the framework: both: the data provider in `LoginTest` (Q62) and `@Parameters("expectedTitle")` in `TestNgFeaturesTest.titleMatchesTheSuiteParameter()` (Q103).

Q65. What goes into `testng.xml`?

The suite and its `<test>` blocks; which classes or packages to run; groups to include or exclude; parameters; listeners; and parallel settings. It is the place to define different runs, such as smoke versus regression, without code changes.

Example:

```
<suite name="OrangeHRM UI Suite" parallel="classes" thread-count="3">
  <parameter name="expectedTitle" value="OrangeHRM" />
  <test name="OrangeHRM UI Tests">
    <groups><run><exclude name="demo" /></run></groups>
    <packages><package name="com.automation.selenium.tests" /></packages>
  </test>
</suite>
```

In the framework: this is `testng.xml`, used for IDE runs. Maven runs the classes directly with the same settings (Q75).

Q66. What can run in parallel in TestNG?

`parallel` can be `methods`, `classes`, `tests` or `instances`, and `thread-count` limits the threads. With Selenium, each thread must have its own driver (`ThreadLocal`, Q45), and tests must not share data.

Example: `parallel="classes" thread-count="3"` runs `LoginTest`, `SystemUsersTest` and `EmployeeTest` at the same time, each in its own thread and browser.

In the framework: classes run in parallel on 3 threads, set in `pom.xml` (`<parallel>classes</parallel>`, `<threadCount>${threads}</threadCount>`) and `testng.xml`; `-Dthreads=1` runs them one after another.

In the lab: `ParallelExecutionTest` runs invocations on a `threadPoolSize` of 3, forces them to overlap, and shows a `ThreadLocal` keeping each thread's value apart; `ThreadLocalDriverTest` does the same with two real browsers.

Q67. Why `parallel=classes` rather than `methods` ?

With `classes`, the methods of one class run one after another on the same thread, while different classes run at the same time. That keeps `priority`, `dependsOnMethods` and any per-class state simple, and you still get one browser per thread.

Example: in `TestNgFeaturesTest`, `adminLogsInAfterThePageCheck` depends on `loginPageIsShown`; with `parallel=classes` they run in order on one thread.

In the framework: `parallel=classes` everywhere. `EmployeeTest` relies on it: its `undelatedEmployeeId` field is safe because one thread runs all its methods.

Q68. What TestNG listeners do you know?

`ITestListener` (a test started, passed, failed or skipped), `ISuiteListener` (suite start and end), `IInvokedMethodListener` (around every method, including configuration methods), `IReporter` (builds a report after the whole run), `IRetryAnalyzer` (decides on retries) and `IAnnotationTransformer` (changes annotations at runtime, for example to add a retry analyzer to every test).

Example:

```
public class ScreenshotListener implements ITestListener {
    @Override
    public void onTestFailure(ITestResult result) { /* take a screenshot */ }
}
```

In the framework: `HtmlReportListener` implements `IReporter`; `RetryOnce` implements `IRetryAnalyzer`. Screenshots are taken in `BaseTest`'s `@AfterMethod` instead of a listener, because the cleanup must run after them.

In the lab: `ListenerTest` with `TestEventsListener`, a small `ITestListener` registered with `@Listeners` (which applies to the whole suite).

Q69. How do you register a listener?

Four ways: `@Listeners(MyListener.class)` on a test class; `<listeners>` in `testng.xml`; `-listener` on the command line; or a ServiceLoader file `META-INF/services/org.testng.ITestNGListener` that names the class. The ServiceLoader file works everywhere (Maven, `testng.xml` and IDE) without being listed anywhere else.

Example: the whole content of the file:

```
com.automation.selenium.report.HtmlReportListener
```

In the framework: [META-INF/services/org.testng.ITestNGListener](#) registers the report listener. It is deliberately not in testng.xml too, or the report would be written twice.

Q70. How do you build a custom HTML report?

Implement `IReporter`. TestNG calls `generateReport(...)` once after the run with every suite result. Collect the passed, failed and skipped results; read each test's steps with `Reporter.getOutput(result)` and any data you stored on the result (such as a screenshot); then write one HTML file.

Example:

```
public void generateReport(List<XmlSuite> xml, List<ISuite> suites, String
outputDirectory) {
    for (ISuite suite : suites) {
        for (ISuiteResult r : suite.getResults().values()) {
            r.getTestContext().getFailedTests().getAllResults(); // also
passed and skipped
        }
    }
}
```

In the framework: [HtmlReportListener](#) writes `selenium-test-report.html`: pass-rate ring, environment, steps, failure message, filtered stack trace, embedded screenshot, search, filters and dark mode. The README shows it.

Q71. How do you retry a failed test?

Implement `IRetryAnalyzer`. TestNG calls `retry(result)` after each failed attempt; return `true` to run the test again. Attach it to one test with `retryAnalyzer`, or to every test with an `IAnnotationTransformer`. Retries also hide real bugs, so log each one and fix tests that need them often.

Example: see the full class in Q102.

In the framework: [RetryOnce](#), attached to [TestNgFeaturesTest.adminLoginIsRetriedOnceOnFailure\(\)](#).

Q72. Where does `testng-failed.xml` come from?

TestNG writes it into its output folder after a run that had failures. It lists only the failed tests (and the configuration methods they need), so you can rerun just those.

Example:

```
mvn test -Dsurefire.suiteXmlFiles=target/surefire-reports/testng-failed.xml
```

In the framework: written to `target/surefire-reports/` after a run with failures, for example `mvn clean test -Pdemo`, which includes the 2 demo tests that fail on purpose.

Q73. Why would a test show as skipped?

Four common reasons: it threw `SkipException`; a method it depends on failed; a configuration method such as `@BeforeMethod` failed, so the tests after it were skipped; or a retry analyzer reran it, and the failed attempt is reported as skipped.

Example: if the browser cannot start in `@BeforeMethod`, every test of that class is skipped rather than failed; the configuration failure is the real error.

In the framework: [ReportDemoTest](#) shows the first two reasons on purpose, and [HtmlReportListener](#) prints the skip reason under each skipped test.

8. Maven, CI and reporting

Q74. How does Maven run TestNG tests?

The Surefire plugin runs tests in Maven's `test` phase. Because `testng` is a dependency, Surefire picks its TestNG provider automatically. It runs every class named `*Test` (or `Test*`, `*Tests`, `*TestCase`), or the suite files listed in `suiteXmlFiles`.

Example:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.5.6</version>
  <configuration>
    <parallel>classes</parallel>
    <threadCount>${threads}</threadCount>
  </configuration>
</plugin>
```

In the framework: this is the Surefire block of [pom.xml](#); it runs the `*Test` classes directly, without a suite file.

Q75. Why might `-Dgroups=smoke` do nothing?

When Surefire runs a suite XML file (`suiteXmlFiles`), the suite decides what runs, and `groups`, `excludedGroups` and `-Dtest` from the command line are ignored. To keep command-line filters working, let Surefire run the classes and put the shared settings (parallel mode, thread count, excluded groups) into the Surefire configuration.

Example: with `<suiteXmlFiles><suiteXmlFile>testng.xml</suiteXmlFile></suiteXmlFiles>` in the pom, `mvn test -Dgroups=smoke` still runs every test in testng.xml.

In the framework: `pom.xml` has no suite file, so `-Dgroups` and `-Dtest` work; `testng.xml` repeats the same settings for IDE runs.

Q76. How do you run one class, one method or one group?

Use Surefire's `test` property for classes and methods, and `groups` for groups. Several values are separated by commas, and `+` joins methods of one class.

Example:

```
mvn clean test -Dtest=LoginTest
mvn clean test -Dtest=LoginTest#adminCanLogIn
mvn clean test -
Dtest=LoginTest#backAfterLogoutStaysLoggedIn+dashboardRequiresLogin
mvn clean test -Dgroups=smoke
mvn clean test -Dtest=LoginTest,EmployeeTest
```

In the framework: all of these work; they are listed under "How to run" in the [README](#), with one group per interview topic (e.g. `-Dgroups=frames`).

Q77. How do you pass settings to tests from the command line?

With `-Dkey=value`. Maven passes it on to the test JVM as a system property, and the framework reads it with `System.getProperty`, falling back to its properties file.

Example:

```
mvn clean test -Dbrowser=edge -Dheadless=true -Dthreads=1
```

In the framework: `Config.value(key)` checks the system property first, so every key of `config.properties` can be overridden; `threads` is a Maven property used by Surefire.

Q78. What is a Maven profile, and when do you use one?

A profile is a named set of properties or plugin settings that you switch on with `-P<id>`. Use it for variations of the build: another environment, an extra group of tests, or CI-only settings.

Example:

```
<profile>
  <id>demo</id>
  <properties>
    <excludedGroups>none</excludedGroups>    <!-- nothing is excluded -->
  </properties>
</profile>
```

In the framework: the `demo` profile in [pom.xml](#). By default `excludedGroups` is `demo`; `mvn clean test -Pdemo` also runs the tests that fail and skip on purpose.

Q79. How do you run Selenium tests in CI?

Use a machine or container that has the browser installed, run the tests headless, and keep the results: publish Surefire's JUnit XML so the CI tool shows each test, and save the HTML report and screenshots as build artifacts. Keep secrets such as passwords in the CI tool's secret store and pass them with `-D`.

Example: a CI step:

```
mvn -B clean test -Dheadless=true
# then save target/surefire-reports/ and target/screenshots/ as artifacts
```

In the framework: no pipeline here on purpose: the [README](#) keeps CI for a separate production-style project. `mvn clean test -Dheadless=true` is all a CI job needs.

Q80. What is Selenium Grid?

Grid runs tests on remote machines. A hub receives each session request and sends it to a node that has the requested browser and operating system, so many tests run at once on many machines. Tests use `RemoteWebDriver` with the hub's URL; Docker images such as `selenium/standalone-chrome` make setup easy.

Example:

```
WebDriver driver = new RemoteWebDriver(new URL("http://localhost:4444"), new
ChromeOptions());
```

In the framework: not used; tests run on the local machine. Adding a `remote` case to [DriverManager.start\(\)](#) would be the only change needed.

Q81. Which reports have you used?

TestNG's built-in reports (`index.html`, `emailable-report.html`), Surefire's JUnit XML for CI, ExtentReports, Allure and custom listeners. Choose by audience: XML for machines, a single shareable HTML file for people.

Example: Allure needs a results folder plus a CLI step to build the report; a custom `IReporter` writes the final HTML file directly at the end of the run.

In the framework: a custom report, `HtmlReportListener`, writes `target/surefire-reports/selenium-test-report.html`; TestNG's own reports are written next to it.

Q82. What should a good UI test report show for a failure?

Enough to understand the failure without rerunning it: which test failed, the steps it ran, the error with expected and actual values, a short stack trace without framework noise, a screenshot of the moment it failed, and the environment (browser, URL, build).

Example: "Demo failure: the dashboard title is checked against a wrong value" → steps 1-2 → `AssertionError: Dashboard title expected [Home] but found [Dashboard]` → screenshot of the dashboard.

In the framework: `HtmlReportListener` shows all of these; `trace()` removes TestNG, reflection and Maven lines from the stack trace. See the failures screenshot in the README.

9. Scenario and troubleshooting questions

Q83. A test passes locally but fails in CI. How do you investigate?

Start from the evidence, then compare environments, then reproduce:

1. Read the failure message, steps and screenshot: did the page not load, was an element missing, or was a value wrong?
2. Compare the environments: headless or not, window size, browser and driver version, network speed, time zone, and the test data on that environment.
3. Run it locally the way CI does (`-Dheadless=true`, same browser) and look for timing gaps, such as an action that does not wait for a load.

Example: a test clicks a menu item that is visible at 1920×1080 but collapsed into a "hamburger" menu in CI's default 800×600 headless window. Fix: set the window size in the options.

In the framework: `DriverManager` sets 1920×1080 for headless and visible runs alike, and `BaseTest` attaches the failure screenshot, so a CI failure can be read from the report.

Q84. How do you deal with flaky tests?

Find the cause before adding a retry. The usual causes: missing waits, stale elements after a re-render, shared or changing test data, tests that depend on each other's order, and an unstable environment. Fix the cause; use a retry only for the environment part, and keep it visible.

Example: a search test failed now and then with `StaleElementReferenceException`, because the table re-rendered while it was read. Reading the table inside one wait (which retries) fixed it for good; a retry would only have hidden it.

In the framework: that is the story of `BasePage.tableRows()` and `wait.ignoring(StaleElementReferenceException.class)` (Q25). `RetryOnce` exists, but only one demonstration test uses it.

Q85. Several tests fail at once with timeouts. What do you check first?

Whether the application is up and fast, before suspecting the tests. Many unrelated tests failing together with timeouts usually means one shared cause: the server, the network or test data. Look at the screenshots first; then call the site yourself.

Example:

```
curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://opensource-demo.orangehrmlive.com/web/index.php/auth/login
```

In the framework: in one run 3 real tests timed out, and their screenshots showed an nginx **504 Gateway Time-out** from the OrangeHRM demo site. A minute later the site answered `200` in about 1 s, and the rerun was green. The README lists this under "Known limitations".

Q86. A click fails with `ElementClickInterceptedException`. Why?

Something covers the element at the moment of the click: a loading spinner, a toast, a cookie banner, a modal's backdrop or a sticky header. Wait for the overlay to go, or scroll the element into view. A JavaScript click is the last resort, because it hides the problem a real user would have.

Example: clicking "Search" while the previous search's spinner is still on the page. Fix: wait for the spinner to disappear first.

In the framework: every search, reset and delete calls `BasePage.waitForLoader()`, and `click()` waits for `elementToBeClickable`. Chrome's password-leak pop-up, another overlay, is switched off in `DriverManager.chromeOptions()`.

In the lab: `SeleniumExceptionsTest.elementClickInterceptedException()` covers the page with an overlay, shows the refused click, waits for the overlay to go and clicks again.

Q87. Tests pass alone but fail in parallel. What is wrong?

Something is shared between threads: a static driver, a page object or element stored in a static field, or several tests changing the same record. Use a `ThreadLocal` driver, create page objects per test, and give each test its own unique data.

Example: two tests both create the employee ID EMP001 ; in parallel, one deletes the other's employee. A random ID per test fixes it.

In the framework: `DriverManager` keeps a `ThreadLocal` driver; pages are created per test; `TestData.newEmployee()` generates a random QA + 8 digit ID.

Q88. How do you test a flow that creates data on a shared environment?

Create your own data with unique values, check it, then delete it in the same test. Also clean up when the test fails halfway, after the failure screenshot so the screenshot still shows the failure, and make the cleanup itself safe: a failing cleanup must never hide the test's real result.

Example:

```
@Override
protected void cleanUp() {                                // called by BaseTest after
every test
    if (undeletedEmployeeId != null) {
        // open PIM, search the ID, delete the employee if it is still there
    }
}
```

In the framework: `EmployeeTest` adds, finds and deletes its own employee and overrides `cleanUp()`. `BaseTest.stopBrowser()` runs screenshot → cleanup → quit, catches cleanup errors, and attaches the cleanup steps to the test's report. Tested by forcing a failure right after the employee was saved: the cleanup deleted it.

10. Questions about your own project

Q89. Walk me through your framework.

Answer top-down in about two minutes: what it tests and with what, how the code is layered, how it runs, how results are reported, and what it covers. Point at one concrete design decision and why you made it.

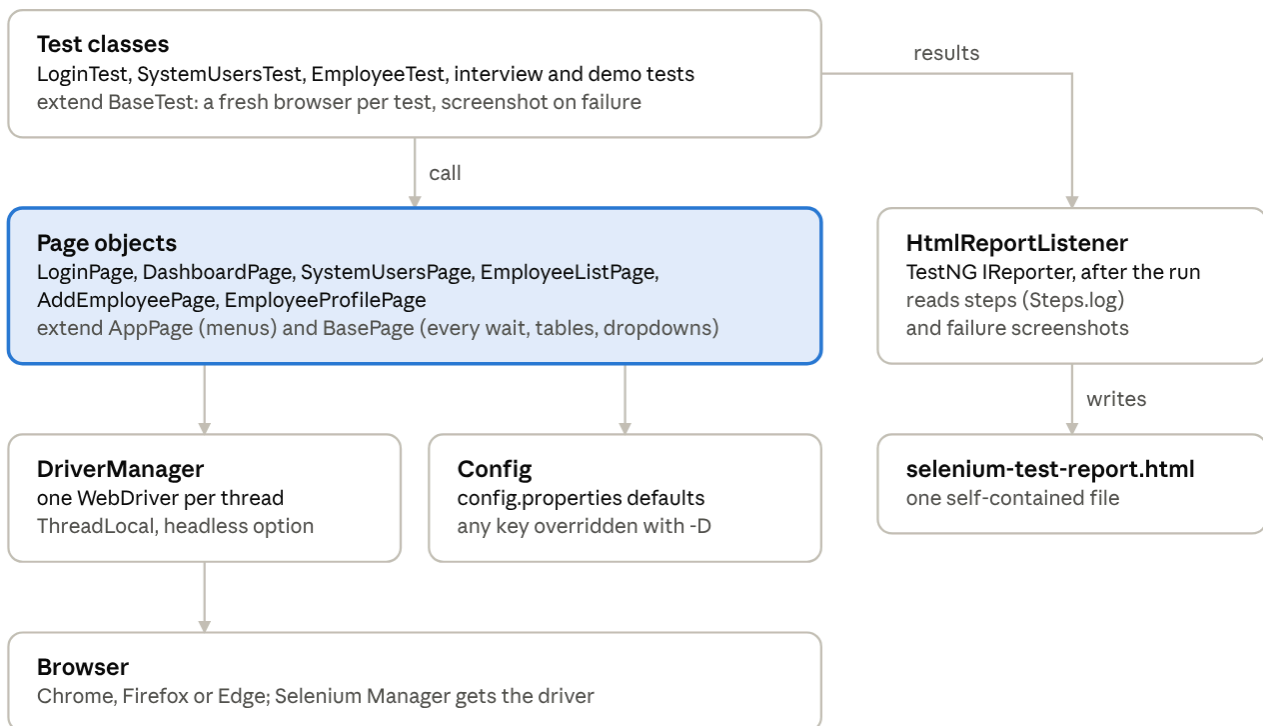
Example answer, with where each part lives:

1. **Stack:** Java 17, Selenium 4.50, TestNG 7.12 and Maven, testing the OrangeHRM demo site (`pom.xml`).
2. **Layers:** test classes extend `BaseTest` and call page objects; the pages extend `AppPage` and `BasePage`, which hold every wait; `DriverManager` and `Config` sit underneath.

3. **Running it:** test classes run in parallel on 3 threads, each test in its own browser; groups, browser, headless mode and thread count are switched from the command line.
4. **Reporting:** a custom `Reporter` , [HtmlReportListener](#) , writes one HTML file with steps, failures and screenshots.
5. **Coverage:** 17 core tests for login, access control, admin user search and the PIM employee life cycle; 17 interview-scenario tests on OrangeHRM; 74 runs of the interview lab (one class per topic on a practice site, plus TestNG and framework examples); plus 4 demo tests that fail and skip on purpose to show the report. A normal run: 108 tests in about 12 minutes, most of it waiting for the two public sites.

In the framework: each item links to its file, and the diagram below shows how the layers fit together; the full file list is in References.

Tests call pages, pages hold every wait, one listener builds the report



selenium-java-framework layers · 7 parts

Tests never touch WebDriver directly. After the run, the report listener reads each test's steps and failure screenshot.

Q90. What would you improve next?

Show that you know the limits of your own work and have a plan. Name concrete next steps and why each matters.

Example answer:

- **CI pipeline:** run `mvn -B clean test -Dheadless=true` on every push and keep the report as an artifact.
- **More browsers:** run Edge and Firefox, locally or through Selenium Grid (a `remote` case in `DriverManager`).
- **Retries for outages only:** apply `RetryOnce` to every test through an `IAnnotationTransformer`, with each retry logged.
- **Faster test data:** create employees through OrangeHRM's API instead of the UI where the UI is not what is being tested.
- **File upload:** done in the lab (Q38); the employee photo upload would add it to the OrangeHRM tests.

In the framework: the [README](#) lists what comes next and the known limitations. CI and Selenium Grid are kept for a separate production-style project, so this repository stays an interview lab.

Other questions to prepare from your own code

- *Why did you write your own report instead of Extent or Allure?* No extra dependency, and one self-contained file that anyone can open and share ([HtmlReportListener](#)).
- *Why are the demo tests excluded by default?* So the normal build stays green; `-Pdemo` includes them ([pom.xml](#) , Q78).
- *Why does `type()` use Ctrl+A and Delete instead of `clear()`?* OrangeHRM's Vue forms keep the old value after `clear()` (Q36, [BasePage.type\(\)](#)).
- *How does the report get the screenshot?* `BaseTest` stores it as the result attribute `HtmlReportListener.SCREENSHOT_ATTRIBUTE`, and the listener embeds it ([BaseTest.attachScreenshot\(\)](#)).
- *Why does the login step mask the password?* Reports get shared; secrets must not end up in them ([LoginPage.mask\(\)](#)).
- *Why does `loginAs` stop with the login error instead of waiting 15 seconds for the dashboard?* A clear message is faster to act on than a timeout (Q28).
- *Why does the logout test reload after Back?* Back can show the browser's cached copy of the dashboard; the reload asks the server, which must answer with the login page (Q8, [LoginTest](#)).

11. Coding scenarios: Selenium

These are the "write the code" questions. Each one is a real, passing test in [SeleniumScenariosTest](#), with the `WebDriver` calls in the page objects. The examples show the core lines.

Q91. Get all items of a menu (or any list of elements) and check them.

`findElements` returns every match; map them to their text. Check with `containsAll` rather than `equals`, so a new menu entry on the site does not break the test, and check that no entry is blank.

Example:

```
// Page: a PageFactory list of every side menu link, read once all are
// visible
@FindBy(css = "a.oxd-main-menu-item")
private List<WebElement> menuEntries;

List<String> items =
    wait.until(ExpectedConditions.visibilityOfAllElements(menuEntries))
        .stream().map(WebElement::getText).map(String::trim).toList();

// Test: the core modules are there, and no entry is blank
assertTrue(items.containsAll(List.of("Admin", "PIM", "Leave", "Time",
    "Dashboard")));
assertTrue(items.stream().noneMatch(String::isBlank));
```

In the framework: `AppPage.menuItems()` reads the list;

`SeleniumScenariosTest.sideMenuListsTheCoreModules()` checks it. On the demo site the menu has 12 entries, from Admin to Buzz.

Q92. Find the broken links on a page.

Selenium can read a link's URL but not its HTTP status, so request each URL with Java's `HttpClient` and treat 400 or above as broken. Read the `href` property, which is always absolute (the attribute can be relative). Collect every broken link before asserting, so one failure lists them all.

Example:

```
Map<String, Integer> broken = new LinkedHashMap<>();
for (String link : dashboard.menuLinks()) {           // href property of each
    menu link
        int status = Links.statusCode(link);           // HttpClient GET,
    follows redirects
        if (status >= 400) {
            broken.put(link, status);
        }
    }
    assertTrue(broken.isEmpty(), "Broken links: " + broken);
```

The HTTP client has no login cookie, so OrangeHRM redirects it to the login page; a 200 after the redirect still proves the link leads somewhere.

In the framework: `AppPage.menuLinks()` collects the URLs, `Links.statusCode()` requests them, and `SeleniumScenariosTest.sideMenuHasNoBrokenLinks()` logs every link with its status in the report. All 12 return 200.

Q93. Read the options of a drop-down that is not a `<select>`.

`Select` only works on `<select>` elements. For a custom drop-down: open it, read the option elements, then close it so the page is left as it was.

Example:

```
By field = inField("User Role", "//div[contains(@class,'oxd-select-text')]");
click(field);                                     // open the list
List<String> options = wait.until(ExpectedConditions
    .visibilityOfAllElementsLocatedBy(By.cssSelector("div[role='listbox']
    div[role='option']"))))
    .stream().map(WebElement::getText).map(String::trim).toList();
click(field);                                     // close it again
assertEquals(options, List.of("-- Select --", "Admin", "ESS"));
```

In the framework: `BasePage.options(label)`, called by `SystemUsersPage.roleOptions()`; checked by `SeleniumScenariosTest.userRoleDropdownOffersAdminAndEss()`.

Q94. Log in using only the keyboard.

The `Actions` class builds a chain of input events: click into the first field, type, press Tab to move on, type, press Enter to submit. Nothing is sent until `perform()`. It also proves the form works for keyboard users.

Example:

```
new Actions(driver)
    .click(usernameField)           // focus the first field
    .sendKeys("Admin")
    .sendKeys(Keys.TAB)            // move to the password field
    .sendKeys("admin123")
    .sendKeys(Keys.ENTER)         // submit the form
    .perform();
```

In the framework: `LoginPage.loginWithKeyboard()`; checked by `SeleniumScenariosTest.adminCanLogInWithTheKeyboard()`.

Q95. Open a new tab, work in it, close it and return.

Store the first window handle before you open anything. `newWindow(WindowType.TAB)` opens a tab and switches to it. After `close()` the driver points at no window until you switch back. Interviewers also like the follow-up: tabs of one browser share the session

cookie, so the new tab is already logged in.

Example:

```
String firstTab = driver.getWindowHandle();
driver.switchTo().newWindow(WindowType.TAB);           // Selenium 4: opens
and switches
new DashboardPage(driver).open();                     // already logged in:
tabs share cookies
assertEquals(driver.getWindowHandles().size(), 2);

driver.close();                                       // closes the current
(second) tab
driver.switchTo().window(firstTab);                  // must switch back
explicitly
```

In the framework: `SeleniumScenariosTest.dashboardOpensInASecondTab()` in [SeleniumScenariosTest](#); [DashboardPage.open\(\)](#) opens the dashboard by its URL.

Q96. Use JavaScript to read what WebDriver cannot.

Cast the driver to `JavascriptExecutor` and run a script; `return` hands a value back as a Java type (String, Long, Boolean, WebElement, List). Here: the page's load state, and the title read a second way to compare with `getTitle()`.

Example:

```
String readyState = (String) ((JavascriptExecutor)
driver).executeScript("return document.readyState;");
assertEquals(readyState, "complete");
```

In the framework: [BasePage.runScript\(\)](#) wraps the cast; [AppPage.readyState\(\)](#) and [titleFromScript\(\)](#) use it; checked by `SeleniumScenariosTest.javascriptReadsThePageState()`.

Q97. Take a screenshot of a single element.

Selenium 4 lets any `WebElement` capture itself, which is useful to compare one widget or attach just the relevant part. Check the result is a real image: every PNG file starts with the bytes `89 50 4E 47`.

Example:

```
byte[] png =
driver.findElement(By.tagName("form")).getScreenshotAs(OutputType.BYTES);
assertEquals(Arrays.copyOf(png, 4), new byte[] { (byte) 0x89, 'P', 'N', 'G'
});
Files.write(Path.of("target", "screenshots", "login-form.png"), png);
```

In the framework: [LoginPage.formScreenshot\(\)](#);

`SeleniumScenariosTest.loginFormScreenshotIsAPng()` checks the bytes and saves `target/screenshots/login-form.png`.

12. Coding scenarios: TestNG

Each TestNG feature below is used on a real OrangeHRM check in [TestNgFeaturesTest](#). Data providers and groups are in `LoginTest`; a skip and a failed dependency are in `ReportDemoTest`.

Q98. Check several things on a page and report every mismatch at once.

Use `SoftAssert`. Each check records its failure and carries on; `assertAll()` at the end fails the test with every mismatch listed. Create a new `SoftAssert` in every test, because it keeps its failures, and never forget `assertAll()`, or the test passes even when checks failed.

Example:

```
SoftAssert soft = new SoftAssert(); // a new one per test:
it keeps state
soft.assertEquals(driver.getTitle(), "OrangeHRM", "Page title");
soft.assertEquals(login.placeholder("Username"), "Username", "Username
placeholder");
soft.assertEquals(login.placeholder("Password"), "Password", "Password
placeholder");
soft.assertEquals(login.buttonText(), "Login", "Button text");
soft.assertAll(); // fails here, listing
every mismatch
```

In the framework: `TestNgFeaturesTest.softAssertChecksTheLoginPage()`, using [LoginPage.placeholder\(\)](#) and `buttonText()`.

Q99. Write a test that passes only when a specific error happens.

`expectedExceptions` names the exception type; `expectedExceptionsMessageRegExp` also checks its message with a regular expression. The test fails if nothing is thrown, a different exception is thrown, or the message does not match.

Example:

```
@Test(expectedExceptions = IllegalStateException.class,
      expectedExceptionsMessageRegExp = ".*Invalid credentials.*")
public void wrongPasswordThrowsTheLoginError() {
    openLoginPage().loginAs("Admin", "wrong-password"); // throws when the
    error banner shows
}
```

In the framework: `TestNgFeaturesTest.wrongPasswordThrowsTheLoginError()`; the exception comes from [LoginPage.loginAs\(\)](#).

Q100. Run one test only after another one passed.

Combine `priority` (the order within a class; lower runs first) with `dependsOnMethods` (the actual condition). If the first test fails, TestNG skips the second rather than failing it, so the report shows one root failure instead of two.

Example:

```
@Test(priority = 1)
public void loginPageIsShown() { ... }

@Test(priority = 2, dependsOnMethods = "loginPageIsShown")
public void adminLogsInAfterThePageCheck() { ... }
```

In the framework: these two methods in `TestNgFeaturesTest`; the failing-dependency case is in [ReportDemoTest](#) (Q58).

Q101. Run the same test several times.

`invocationCount = 2` runs it twice. Each run gets its own `@BeforeMethod`, and therefore its own browser, and the report lists each run separately. Use it to expose a check that only sometimes passes.

Example:

```
@Test(invocationCount = 2)
public void wrongPasswordIsRejectedEveryTime() {
    LoginPage login = openLoginPage().submit("Admin", "wrong-password");
    assertEquals(login.errorMessage(), "Invalid credentials");
}
```

In the framework: `TestNgFeaturesTest.wrongPasswordIsRejectedEveryTime()`; the report shows it twice.

Q102. Retry a test that failed because of the environment.

Implement `IRetryAnalyzer`. TestNG calls `retry()` after each failed attempt; `true` runs the test again. TestNG keeps one analyzer per test method, so a field can count the attempts. The failed attempt shows as skipped in the report, so retries stay visible.

Example:

```
public class RetryOnce implements IRetryAnalyzer {
    private static final int MAX_RETRIES = 1;
    private int retries;                // attempts used for
    this test method

    @Override
    public boolean retry(ITestResult result) {
        if (retries < MAX_RETRIES) {
            retries++;
            return true;                // run the test again
        }
        return false;                  // report the failure
    }
}

@Test(retryAnalyzer = RetryOnce.class)
public void adminLoginIsRetriedOnceOnFailure() { ... }
```

A retry hides a real bug just as well as a site outage, so log every retry and fix the cause of repeated ones.

In the framework: [RetryOnce](#) (it also prints each retry with the failure), used by `TestNgFeaturesTest.adminLoginIsRetriedOnceOnFailure()`.

Q103. Pass a value from testng.xml into a test.

`@Parameters` reads `<parameter>` tags from the suite file. When the test runs without that file (Maven running the classes directly, or the IDE running one class), `@Optional` supplies the default instead of failing with a missing-parameter error.

Example:

```
<parameter name="expectedTitle" value="OrangeHRM" />
```

```

@Parameters("expectedTitle")
@Test
public void titleMatchesTheSuiteParameter(@Optional("OrangeHRM") String
expectedTitle) {
    openLoginPage();
    assertEquals(driver.getTitle(), expectedTitle);
}

```

In the framework: the parameter is in [testng.xml](#); the test is

```
TestNgFeaturesTest.titleMatchesTheSuiteParameter().
```

Q104. When do you use `@Test(timeout = ...)` in a Selenium suite?

`timeout` fails a test that runs longer than the given milliseconds. In a Selenium suite it is a safety net against a test that hangs, for example a page load that never ends. It does not replace explicit waits: a wait fails at the exact step with a precise message, while `timeout` only says the whole test took too long. If your driver lives in a `ThreadLocal`, check that the timed test still sees the browser that `@BeforeMethod` started.

Example:

```

@Test(timeout = 60_000)                // fail if the whole test takes more than
60 s
public void loginFinishesWithinOneMinute() {
    assertTrue(loginAsAdmin().isDisplayed());
}

```

In the framework: [TestNgFeaturesTest.loginFinishesWithinOneMinute\(\)](#). With TestNG 7.12 and Surefire's `parallel=classes`, the timed test uses the `ThreadLocal` browser from [DriverManager](#) normally; this was checked with a probe test before the test was added.

References

Framework files

Every file below is on GitHub at [niranjankagri/selenium-java-framework](#) (branch `main`).

File	What it shows	Questions
pages/BasePage.java	PageFactory setup, explicit waits, stale handling, spinner wait, label-based locators, custom drop-downs, tables, <code>type()</code> instead of <code>clear()</code> , JavaScript	Q3, Q7, Q8, Q14-Q19, Q21, Q23-Q28, Q30, Q35, Q36, Q39, Q42, Q44, Q84, Q86, Q93, Q96
pages/AppPage.java	Side menu, user menu, list of elements, link URLs, page state through JavaScript	Q39, Q43, Q44, Q91, Q92, Q96
pages/LoginPage.java	@FindBy fields (by name, CSS and tag), @CacheLookup demo, clear login error, keyboard login with <code>Actions</code> , element screenshot, masked password	Q3, Q7, Q12, Q13, Q28, Q34, Q39, Q41, Q42, Q50, Q94, Q97-Q99
pages/DashboardPage.java	Opening a page by URL, with and without waiting	Q95
pages/SystemUsersPage.java	Fluent filters, drop-down options, table columns	Q18, Q30, Q40, Q43, Q93
pages/EmployeeListPage.java	Clicking a button in a table row found by its text; in-page delete dialog	Q16, Q19, Q31
pages/AddEmployeePage.java	Locators by name, saving with and without validation errors	Q13
pages/EmployeeProfilePage.java	Custom wait condition (lambda)	Q27
driver/DriverManager.java	ThreadLocal driver, browser factory, options, headless, Selenium Manager	Q1, Q2, Q4, Q6, Q9, Q10, Q22, Q45, Q49, Q80, Q83, Q87, Q104

File	What it shows	Questions
config/Config.java	Properties file with <code>-D</code> overrides; singleton	Q9, Q46, Q49, Q77
report/HtmlReportListener.java	Custom <code>IReporter</code> HTML report with steps, screenshots and skip reasons	Q37, Q50, Q68, Q70, Q73, Q81, Q82
utils/Steps.java	Test steps through <code>Reporter.log</code>	Q50
utils/Links.java	HTTP status of a link with <code>HttpClient</code>	Q11, Q92
META-INF/services/org.testng.ITestNGListener	Listener registration through <code>ServiceLoader</code>	Q69
base/BaseTest.java	Browser per test, failure screenshot, cleanup hook, <code>alwaysRun</code>	Q37, Q48, Q49, Q52-Q54, Q83, Q88
base/RetryOnce.java	Retry analyzer	Q68, Q71, Q84, Q102
data/TestData.java	Shared data provider, unique generated employees	Q47, Q62, Q63, Q87
tests/LoginTest.java	Groups, data provider, Back + reload after logout, login required	Q8, Q55, Q62, Q63
tests/SystemUsersTest.java	Fluent page calls; checks that do not depend on shared data	Q43
tests/EmployeeTest.java	Create, find and delete own data; cleanup after a failure	Q48, Q67, Q88
tests/SeleniumScenariosTest.java	The 7 Selenium coding scenarios, plus <code>@FindBy</code> vs <code>@CacheLookup</code> after a reload	Q3, Q6, Q33, Q42, Q91-Q97

File	What it shows	Questions
tests/TestNgFeaturesTest.java	SoftAssert, expectedExceptions, priority, dependsOnMethods, invocationCount, timeOut, retryAnalyzer, Parameters	Q56, Q59, Q61, Q64, Q98–Q104
tests/ReportDemoTest.java	Failures and skips on purpose	Q58, Q60, Q73, Q100
pom.xml	Surefire, parallel classes, excluded groups, demo profile	Q51, Q55, Q66, Q74, Q75, Q78
testng.xml	Suite for the IDE, parameter, excluded group	Q55, Q65, Q66, Q103
config.properties	Default settings	Q46, Q77
README.md	How to run, test list, latest results, known limitations	Q76, Q79, Q90

Interview lab files

One example class per topic, in [src/test/java/com/automation/selenium/interview/](#). Each class starts with the interview question and the answer to give. Selenium topics run on [the-internet.herokuapp.com](#); TestNG and framework topics need no site.

File	What it shows	Questions
PracticeSiteTest.java	Base class: opens a practice page, reloads a broken one (logged)	Q83, Q84
locators/LocatorStrategiesTest.java	All eight By strategies, XPath axes, relative locators, a dynamic locator	Q12, Q14, Q16, Q17, Q20
synchronization/ImplicitWaitTest.java	No wait vs implicit wait	Q21, Q22

File	What it shows	Questions
synchronization/ExplicitWaitTest.java	Presence vs visibility, loading indicator, enabled input	Q23, Q24, Q26
synchronization/FluentWaitTest.java	FluentWait, custom condition with a message	Q27
exceptions/SeleniumExceptionsTest.java	Ten exceptions reproduced and handled	Q7, Q25, Q28, Q31, Q32, Q85, Q86
alerts/AlertHandlingTest.java	Alert, confirm, prompt	Q31
frames/IframeHandlingTest.java	Nested frames, <code>parentFrame</code> , <code>defaultContent</code> , <code>iframe</code> by element	Q32
windows/MultipleWindowTest.java	New window by handle set difference and by title	Q33
dropdowns/SelectDropdownTest.java	<code>Select</code> by text, value and index; disabled option	Q29
elements/CheckboxTest.java	Read the state, click only when it must change	Q39
elements/WebTableTest.java	Rows as maps, totals, columns by class, <code>./</code> inside a row	Q19
actions/ActionsClassTest.java	Hover, right-click, key combinations, slider	Q34
javascript/JavaScriptExecutorTest.java	Arguments and return types, scrolling, the JavaScript-click risk	Q35
cookies/CookieTest.java	Add, read, delete cookies; skip the login with a saved session cookie	—
upload/FileUploadTest.java	<code>sendKeys</code> with the absolute path	Q38
testng/AnnotationOrderTest.java	The order of all configuration annotations; <code>alwaysRun</code>	Q52, Q54

File	What it shows	Questions
testng/ListenerTest.java	An <code>ITestListener</code> registered with <code>@Listeners</code>	Q68, Q69
testng/DataProviderVariantsTest.java	Iterator, Method-aware and parallel data providers	Q62
testng/ParallelExecutionTest.java	<code>threadPoolSize</code> and a <code>ThreadLocal</code> under real parallel load	Q45, Q66
testng/DependencyTest.java	<code>dependsOnGroups</code> , <code>enabled = false</code>	Q58
framework/ThreadLocalDriverTest.java	Two threads, two browser sessions; <code>quit()</code> empties the slot	Q45, Q87
framework/ConfigOverrideTest.java	<code>-D</code> overrides the properties file; a missing setting fails fast	Q46, Q77

Official documentation

- [Selenium: Waiting Strategies](#) — explicit, implicit and fluent waits; the warning not to mix implicit and explicit waits (Q21–Q28).
- [Selenium Manager](#) — automatic driver management, included since Selenium 4.6 (Q4).
- [TestNG documentation](#) — annotations, groups, data providers, parallelism and time-outs, rerunning failed tests, listeners (Q51–Q73, Q98–Q104).
- [Maven Surefire: Using TestNG](#) — groups, `excludedGroups` and parallel settings in the pom (Q74–Q78).